

Shoulder Recognition Algorithm

Documentation

Real Galaxy Shoulder Quantification v0.69

Field	Value
Script	Real Galaxy Shoulder Quantification v0.69.py
Location	C:\Users\gordo\Documents\Github\PythonScripts\Shoulder Recognition Erwin\Real Galaxy Shoulder Quantification v0.69.py
Document date	2026-06-19
Documentation scope	High-level overview, data sources, processing workflow, outputs, parameters, and detailed code behavior.

High-Level Overview

This script runs a shoulder-quantification workflow on observed galaxy bar major-axis profiles. Its main purpose is to identify paired shoulder structures in bar profiles from the Erwin, Debattista, and Anderson 2023 classification sample, using S4G 3.6 micron FITS images and a locally generated geometry manifest to construct the required profiles when profile files are not already available.

For each classified galaxy with the required catalog, geometry, and image data, the code extracts a deprojected bar-major-axis intensity profile, normalises radius by the deprojected bar radius, smooths the analysed profile with a low-pass Butterworth filter, computes first and second derivatives and radius-of-curvature structure, then searches for left and right shoulder candidates inside the bar.

- Input data are external to this folder and are mostly hard-coded under the MSc research Dropbox path.
- The analysis window is restricted to $|x/R_{\text{bar}}| < 1.8$, while accepted shoulders must lie inside $|x/R_{\text{bar}}| < 1$.
- By default, a galaxy is counted as having shoulders only when both left and right shoulders are accepted.

- The script writes constructed profile caches, diagnostic plots, missing-data reports, measurement tables, and a run summary.

Data Sources and Dependencies

The script combines local repository helper modules with external research data. The repository supplies geometry and plotting utilities; the measured galaxy data and FITS images live in a hard-coded Dropbox research folder.

Source	Purpose
Erwin catalog data	Reads s4gbars_table.dat for bar and galaxy properties, scrambled_map.txt to map classification IDs to galaxy names, and classifications_pe.txt plus classifications_vd_revised.txt to define the classified sample.
S4G geometry manifest	Reads Erwin_s4g_image_downloader/geometry_output/s4g_image_geometry_manifest.csv for image centres, disk PA, inclination, bar PA, bar semi-major axis, pixel scale, CRPIX fallback values, and image path metadata.
S4G FITS images	Loads 3.6 micron images from each manifest image_path or from D:\Dropbox\Public Documents\UCLAN\MSc Research\Erwin\s4g_images_36um\<galaxy>.phot.1.fits.
Helper modules	Imports angle_utils from Erwin_barprofiles_paper_GB_working_copy and plot_s4g_isophote_axes from Erwin_s4g_image_downloader for PA rectification, deprojection, and profile extraction utilities.
Scientific Python stack	Uses numpy, pandas, scipy.signal, astropy.io.fits, scikit-image profile_line, matplotlib, csv, math, os, sys, and warnings.

Important hard-coded paths

Name	Value / meaning
research_folder	D:\Dropbox\Public Documents\UCLAN\MSc Research

Name	Value / meaning
erwin_repo	<research_folder>\Erwin\perwin-barprofiles_paper-a7cd6f5
erwin_data_folder	<erwin_repo>\data
manifest_file	<repo>\Erwin_s4g_image_downloader\geometry_output\s4g_image_geometry_manifest.csv
image_folder	<research_folder>\Erwin\s4g_images_36um
output_folder	<research_folder>\Shoulder_Recognition_Erwin
plots_folder	<output_folder>\plots
profiles_folder	<output_folder>\profiles

Processing Workflow

1. Initialise paths, create output, plots, and profiles folders, and add local helper-module directories to sys.path.
2. Read the Erwin S4G bar table, the scrambled-name map, the two classification files, and the S4G image geometry manifest.
3. Build a sorted classified-galaxy list by combining Peter Erwin and Victor Debattista revised classifications, excluding unknown classifications marked with '?'.
 4. For each classified galaxy, require a catalog row and manifest row, then construct a bar-major-axis profile from the FITS image.
 5. Write a missing-data text report and CSV for galaxies that cannot be analysed.
 6. Loop over all galaxies with usable profiles, interpolate NaNs, normalise the analysed profile, smooth it, calculate derivatives and radius of curvature, and search for left and right shoulder candidates.
 7. Reject shoulders that are too close to the centre, outside the bar, too thin, too steep, too noisy, one-sided when paired shoulders are required, or overlapping.
 8. Save one diagnostic PNG per analysed galaxy and write the final shoulder measurements in NPY and CSV formats plus a plain-text run summary.

Profile construction

`construct_profile_from_manifest` uses the manifest geometry to locate the galaxy centre and bar orientation. It loads the FITS image, falls back to CRPIX coordinates if the manifest centre is outside the image, chooses a profile radius based on the bar semi-major axis and pixel scale, and extracts a profile along the bar major axis with a profile width of 3 pixels.

The extracted radii are converted from pixels to arcseconds and deprojected using the disk PA and inclination. The bar semi-major axis is also deprojected so later analysis can express radius as x/R_{bar} . A minor-axis profile is extracted only to estimate robust plotting limits from finite positive intensities.

Shoulder detection

Within each galaxy, the full extracted profile is kept for plotting, but the analysis uses only $|x/R_{\text{bar}}| < 1.8$. The analysed raw intensity profile is interpolated where NaNs occur, normalised to 0..1, and smoothed with a second-order low-pass Butterworth filter. The cutoff is calculated from the number of profile samples and profile span rather than fixed globally.

The algorithm then calculates the first derivative, second derivative, and radius of curvature. Candidate shoulders are found from derivative extrema near each side of the bar: left-side candidates use first-derivative minima and right-side candidates use first-derivative maxima, ordered from the galaxy centre outward. Nearest radius-of-curvature minima and maxima mark the clavicle and shoulder boundaries.

Control Parameters

Parameter	Default	Role
allow_1_sided_shoulders	False	Requires a left and right shoulder pair for a positive shoulder system.
profile_width	3	Pixel width used when extracting major/minor-axis profiles from FITS images.
thin_be_gone	0.05	Rejects shoulders whose inner-to-outer width is too small in x/R_{bar} units.
peaks_max	12	Rejects profiles with too many first-derivative extrema inside the bar.

Parameter	Default	Role
too_close	0.2	Rejects candidate shoulder clavicles closer than 0.2 R_{bar} to the centre.
x_cutoff_vs_bre	1.8	Limits the algorithmic analysis window to $ x/R_{\text{bar}} < 1.8$.
slope_cutoff	0.35	Candidate shoulder derivative extrema must have absolute slope below this value.
bar_extent	1	Defines the bar boundary in normalised radius after x is divided by R_{bar} .

Outputs

Output	Description
profiles/<galaxy>_bar-major-axis_profile.dat	Constructed cache of deprojected bar-major-axis radius in arcsec and raw S4G 3.6 micron intensity. Three commented header rows are written.
plots/<galaxy>.png	Diagnostic plot showing the semilog major-axis profile, smoothed curve, derivative overlays, residuals, bar boundary markers, and shoulder annotations.
missing_data_components.txt	Plain-text report explaining which required components are missing or unusable for each skipped galaxy.
missing_data_components.csv	CSV version of the missing-data report with galaxy and reason columns.
shoulder_measurements.npy	Structured NumPy array containing final shoulder measurements.
shoulder_measurements.csv	CSV version of the structured shoulder table.
run_summary.txt	Run-level summary of classified galaxies, usable galaxies, skipped galaxies, shoulder systems found, and output locations.

Measurement columns

Column	Meaning
galaxy	Galaxy name.
clav_left / clav_right	Accepted left/right shoulder clavicle location in x/R_{bar} units, or NaN if rejected.
left_inner / right_inner	Inner boundary of the accepted shoulder, usually the inner radius-of-curvature minimum around the clavicle.
left_outer / right_outer	Outer boundary of the accepted shoulder, selected from the next radius-of-curvature structure outward.
left_slope / right_slope	First-derivative value at the accepted clavicle candidate.
left_clav_inner / left_clav_outer	Radius-of-curvature minima bracketing the left clavicle.
right_clav_inner / right_clav_outer	Radius-of-curvature minima bracketing the right clavicle.

Detailed Code Documentation

Module setup

The script starts by forcing matplotlib to use the Agg backend, which allows figures to be saved without an interactive display. It then builds PROJECT_ROOT from the script location and appends two sibling folders to sys.path so angle_utils and plot_s4g_isophote_axes can be imported.

- BARPROFILES_DIR: expected to contain angle_utils.py.
- S4G_PLOTTER_DIR: expected to contain plot_s4g_isophote_axes.py.
- The local function required_geometry and profile_at_pa are defined but the active profile-construction path uses corresponding helpers from plot_s4g_isophote_axes.

Utility functions

Function	Behavior
find_nearest(array, value)	Converts the input to a NumPy array and returns the element whose absolute difference from value is smallest. Used to connect derivative extrema to nearest curvature features and profile bins.
read_s4g_table(filename)	Reads whitespace-delimited s4gbars_table.dat with comment lines ignored, applies a fixed 37-column schema, and returns a pandas DataFrame.
read_descramble_map(filename)	Reads scrambled_map.txt, skipping blank/comment lines, and maps integer scrambled IDs to galaxy names.
read_classified_galaxies(filename, descramble_map)	Reads a classification file, keeps rows with a non-'?' classification, resolves the scrambled ID to a galaxy name, and returns the list of classified galaxies.
read_manifest(filename)	Reads the geometry manifest CSV into a dictionary keyed by galaxy name.
parse_float(value)	Safely converts manifest fields to finite floats; blank, invalid, and non-finite values become None.
pa_endpoint(pa_deg, radius)	Converts a position angle and radius into an x/y endpoint offset. Defined for profile extraction geometry.
profile_at_pa(data, xc, yc, pa_deg, radius_pix, width)	Extracts a line profile through a 2D image using skimage.measure.profile_line, averaging over the requested linewidth and returning pixel radii plus intensity values.
required_geometry(row)	Parses essential geometry fields from a manifest row, rectifies disk and bar PAs to 0..180 degrees, and returns None if required values are missing.
construct_profile_from_manifest(galaxy, manifest_row)	Builds the profile file, deprojected bar radius, and plot scale for one galaxy, or returns a reason string if the galaxy cannot be processed.

Function	Behavior
write_missing_data_report(missing_rows, filename)	Writes a human-readable missing-data report that lists required components and per-galaxy failure reasons.

Main sample assembly

After function definitions, the script immediately executes the analysis at module level. It reads the catalog, descramble map, manifest, and both classification files, then constructs `classified_galaxies` as the sorted unique union of the Peter Erwin and Victor Debattista revised classified samples.

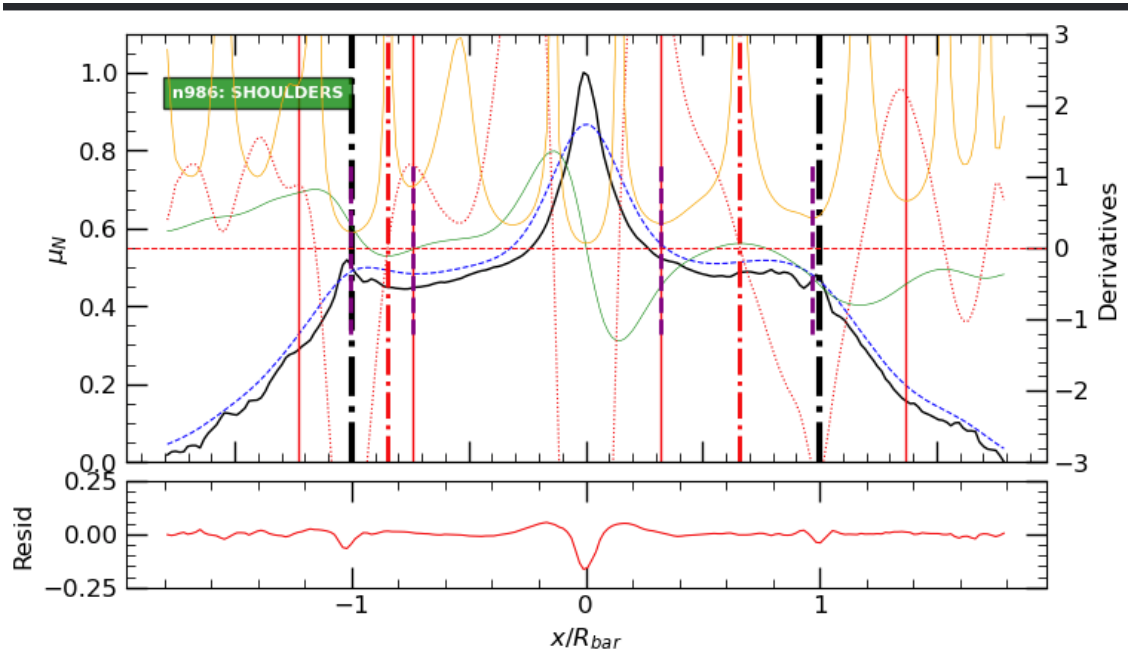
For each galaxy, the code verifies an S4G catalog row, a manifest row, a usable FITS image/profile, and a positive finite deprojected bar radius. Successful galaxies are stored in `available_rows` with galaxy name, `bar_radius`, `profile_file`, and `profile_scale`; failures are stored in `missing_data` with an explanatory reason.

Analysis loop details

- Loads each generated profile with `np.loadtxt`, skipping the three commented header rows.
- Normalises radius by the deprojected bar radius and keeps raw intensity for semilog plotting.
- Repairs leading NaNs by copying the first finite value, then linearly interpolates remaining NaNs with `pandas`.
- Applies the analysis mask $|x/R_{\text{bar}}| < 1.8$ and normalises the analysed intensity values to 0..1.
- Computes a Butterworth cutoff from sample count and x-span, then smooths the normalised profile with `signal.filtfilt`.
- Calculates first derivative, raw first derivative, second derivative, raw second derivative, and radius of curvature.
- Flags a profile as too noisy if the count of first-derivative extrema inside $|x| \leq 1$ exceeds `peaks_max`.
- Searches left and right candidates separately, starting from candidate extrema closest to $x = 0$ and moving outward.
- Accepts only shoulders inside the bar, away from the centre, below the slope cutoff, wider than `thin_be_gone`, and not in a too-noisy profile.
- Rejects final paired shoulders if left and right shoulder regions overlap or are within one bin of the centre.

Plot contents

Each diagnostic PNG has a two-panel layout. The upper panel shows the full raw major-axis intensity profile on a logarithmic y-axis, the smoothed analysed profile over the selected analysis window, derivative overlays on a twin y-axis, bar-boundary markers at $x/R_{bar} = \pm 1$, and red/purple shoulder markers when accepted. The lower panel shows the residual between the smoothed normalised profile and the normalised raw analysed profile.



Example diagnostic output image included with the script folder.

Operational Notes and Maintenance Risks

- The script is not command-line parameterised; changing data locations or thresholds requires editing global variables in the source file.
- The code executes at import time. Importing it from another module would run the full analysis unless the main workflow is later moved behind an `if __name__ == '__main__':` guard.
- Several functions duplicate concepts available in `plot_s4g_isophote_axes`, and `construct_profile_from_manifest` currently calls the imported helper versions for geometry/profile work.
- The profile interpolation step includes a note saying it needs discussion, so NaN handling should be treated as an explicit scientific assumption.

- The shoulder decision depends strongly on smoothing, extrema counts, and slope thresholds. These are documented above because they are the most important reproducibility controls.
- The output folder is outside the repository, so generated scientific outputs are not automatically versioned with the code unless copied or tracked separately.

Suggested future improvements

- Add argparse options or a small configuration file for data paths, output path, and shoulder thresholds.
- Move the executable workflow into a main() function guarded by if __name__ == '__main__'.
- Write a compact run manifest alongside each result CSV that records parameter values, script version, and input file timestamps.
- Consider replacing print-only progress with logging so rejected candidates can be persisted per galaxy.
- Add unit tests around parse_float, manifest validation, profile construction failure modes, and shoulder candidate rejection logic.